Algorithm for File Updates in Python

Project description

In this project, I worked with a scenario involving a healthcare organization that manages access to sensitive patient information. The organization uses an allow list containing IP addresses that are authorized to access a restricted network. Because employees' access permissions may change, the organization also maintains a remove list identifying IP addresses that should no longer be permitted.

My task was to develop a Python algorithm that automatically updates the allow list by removing IP addresses that appear on the remove list. The algorithm reads the existing file, processes its contents, identifies addresses that must be removed, and writes the updated information back to the original file. This process helps maintain accurate access permissions and supports the principle of least privilege.

Open the file that contains the allow list

The first step of the algorithm is to open the file containing the IP addresses that are currently authorized to access the restricted network.

I assigned the filename `allow_list.txt` to a variable called `import_file`. Storing the filename in a variable makes it easier to reference the file throughout the algorithm without repeatedly typing its name.

```python
import_file = "allow_list.txt"

with open(import_file, "r") as file:
```

I used the `with` statement together with the built-in `open()` function to access the file. The `open()` function takes two arguments: the filename stored in `import_file` and the mode `"r"`, which specifies that the file should be opened for reading.

The keyword `as` assigns the opened file to the variable `file`, allowing me to work with its contents inside the `with` block.

An important advantage of using the `with` statement is that Python automatically closes the file when the block finishes executing. This helps manage system resources and avoids leaving files unnecessarily open.

Read the file contents

After opening the file, I needed to retrieve the IP addresses stored in it.

To accomplish this, I used the `.read()` method:

```python
ip_addresses = file.read()
```

The `.read()` method reads the entire contents of the file and returns them as a string. I assigned this string to the variable `ip_addresses`.

At this stage, the variable contains the authorized IP addresses as text rather than as individual list elements.

Reading the file is necessary because the algorithm must examine the existing IP addresses before determining which ones should be removed.

Convert the string into a list

After reading the file, I converted the string containing the IP addresses into a Python list.

I used the `.split()` method:

```python
ip_addresses = ip_addresses.split()
```

The `.split()` method separates a string into individual elements based on whitespace, including spaces and newline characters.

For example, if the file contains multiple IP addresses separated by newline characters, the method converts them into individual elements of a list.

This conversion is important because Python lists allow individual elements to be accessed, compared, and removed.

I reassigned the result to the same variable, `ip_addresses`, because the algorithm no longer needed the original string representation.

By converting the contents into a list, I could use a loop and conditional statements to identify IP addresses that should no longer have access to the restricted network.

Iterate through the remove list

The organization maintains a separate list called `remove_list`, which contains the IP addresses that must be removed from the allow list.

To examine each address in this list, I used a `for` loop:

```
Python
for element in remove_list:
```

The `for` keyword creates a loop that iterates through a sequence of elements.

In this algorithm, `remove_list` is the sequence being examined, while `element` is the loop variable representing the current IP address.

During each iteration, Python assigns the next IP address from `remove_list` to `element`.

This approach allows the algorithm to process every address that needs to be checked without requiring a separate statement for each one.

The loop also makes the algorithm reusable because it can process removal lists containing different numbers of IP addresses.

Remove IP addresses that are on the remove list

Inside the `for` loop, I added a conditional statement to determine whether each IP address from `remove_list` exists in the current allow list.

I used the following code:

```
Python
for element in remove_list:
    if element in ip_addresses:
```

```
        ip_addresses.remove(element)
```

The `if` statement evaluates whether the current value of `element` is present in the `ip_addresses` list.

The `in` operator performs a membership check and returns `True` when the specified IP address exists in the list.

If the condition is true, the `.remove()` method deletes that IP address from `ip_addresses`.

If the IP address is not present, the condition evaluates to `False`, and the algorithm continues to the next element without attempting to remove it.

This conditional check is important because calling `.remove()` on a value that does not exist in a list would raise a `ValueError`.

The `.remove()` method removes the first occurrence of a matching element. In this scenario, the allow list does not contain duplicate IP addresses, so calling `.remove()` once is sufficient to remove each matching address.

After the loop finishes, the `ip_addresses` list contains only the IP addresses that should remain authorized.

Update the file with the revised list of IP addresses

After removing the unauthorized IP addresses, the final step was to save the updated allow list back to the original file.

Because the `.write()` method expects a string rather than a list, I first converted the updated list into a string using the `.join()` method.

Python
```
ip_addresses = "\n".join(ip_addresses)
```

The `.join()` method combines the elements of the `ip_addresses` list into a single string.

I applied `.join()` to the newline character `"\n"`, which places each IP address on a separate line.

This preserves a clear and organized format in the updated text file.

Next, I opened the original file again, this time using write mode:

```python
with open(import_file, "w") as file:
    file.write(ip_addresses)
```

The `"w"` argument instructs Python to open the file for writing. Unlike read mode, write mode replaces the existing contents of the file.

The `.write()` method then saves the revised string of authorized IP addresses to `allow_list.txt`.

I used another `with` statement to ensure that the file is automatically closed after the updated information has been written.

As a result, the original allow list is replaced with an updated version that excludes the IP addresses identified for removal.

Summary

In this project, I developed a Python algorithm to automate the process of updating an IP address allow list used to control access to a restricted network. The algorithm uses the `with` statement and `open()` function to access the file, while the `.read()` method retrieves its contents. It then applies `.split()` to convert the stored IP addresses into a list that can be processed.

Using a `for` loop, a conditional statement, and the `.remove()` method, the algorithm identifies and removes IP addresses that are no longer authorized. Finally, the `.join()` and `.write()` methods convert the revised list back into a string and save it to the original file.

This project demonstrates my ability to work with Python file operations, lists, loops, conditional statements, and string manipulation. It also illustrates how Python automation can support access control management, reduce repetitive administrative work, and help organizations maintain accurate security configurations.